ASSIGNMENT -SOCKET PROGRAMMING

1. Socket Programming: Web Server

In this lab, you will learn the basics of socket programming for TCP connections in Python: how to create a socket, bind it to a specific address and port, as well as send and receive a HTTP packet. You will also learn some basics of HTTP header format.

You will develop a web server:

- that handles one HTTP request at a time.
- accepts and parses the HTTP request,
- get the requested file from the server's file system, create an HTTP response message consisting of the requested file preceded by header lines, and then send the response directly to the client.
- If the requested file is not present in the server, the server should send an HTTP "404 Not Found" message back to the client.

Code

Below you will find the skeleton code for the Web server. You are to complete the skeleton code. The places where you need to fill in code are marked with `#Fill in start` and `#Fill in end`. Each place may require one or more lines of code.

Running the Server

Put an HTML file (e.g., HelloWorld.html) in the same directory that the server is in. Run the server program. Determine the IP address of the host that is running the server (e.g., 128.238.251.26). From another host, open a browser and provide the corresponding URL. For example:

http://128.238.251.26:6789/HelloWorld.html or http://localhost/80 or http://localhost/<htmlfilename>

'HelloWorld.html' is the name of the file you placed in the server directory. Note also the use of the port number after the colon. You need to replace this port number with whatever port you have used in the server code. In the above example, we have used the port number 6789. The browser should then display the contents of HelloWorld.html. If you omit ":6789", the browser will assume port 80 and you will get the web page from the server only if your server is listening at port 80.

Then try to get a file that is not present at the server. You should get a "404 Not Found" message.

NOTE: Python is indent sensitive. Write carefully

Skeleton Python Code for the Web Server

```
#import socket module

from socket import *

serverSocket = socket(AF_INET, SOCK_STREAM)

#Prepare a sever socket

#Fill in start

#Fill in end

while True:

    #Establish the connection

    print 'Ready to serve...'

    connectionSocket, addr =    #Fill in start                #Fill in end

    try:

        message =    #Fill in start           #Fill in end

        filename = message.split()[1]

        f = open(filename[1:])

        outputdata = #Fill in start        #Fill in end

        #Send one HTTP header line into socket

        #Fill in start

        #Fill in end

        #Send the content of the requested file to the client

         for i in range(0, len(outputdata)):

             connectionSocket.send(outputdata[i])

         connectionSocket.close()

    except IOError:

        #Send response message for file not found

        #Fill in start

        #Fill in end

        #Close client socket

        #Fill in start

        #Fill in end
```

```
serverSocket.close()
```

How to run Client server program:

```
# start the server:
$ python server.py
# Don’t terminate the server program
# start the client:
$ python client.py
```

Q.1 Read the instructions and run the simple client server program given as an example.(Client.py, server.py).

Q.2 Execute a client/server program for adding a two integer numbers requested by the client and evaluated at server and get back result at the client.

Q.3 Build a server capable of receiving text messages from clients. The server should print these text messages on the standard output, but should not print any other messages such as debug information. From the server perspective, a message corresponds to the data received from a particular client during a communication session with that client. The server should be listening for text messages to a port known to the clients. The server should be able to queue connections from multiple clients i.e., it will handle the client connections sequentially, but should be able to accept connections from multiple clients. After servicing the current client to completion, it should proceed to the next. If multiple clients simultaneously try to send text messages to the server, the server should print them one at a time (in any order).

Q.4. The Server application has to support at least five clients simultaneously. Server accepts strings from clients (even multiple strings from each client) and replies with reverse strings. For example, when client sends “JIIT”, Server replies with “TIIJ”. Both server and client(s) have to output both sending & receiving strings on the terminal. The server and client processes should be run on different machines